Latest updates: https://dl.acm.org/doi/10.1145/3696630.3734200

INVITED-TALK

Can the Rising Tide of Software Supply Chain Attacks Raise All Software Engineering Boats?

LAURIE WILLIAMS, NC State University, Raleigh, NC, United States

SIVANA HAMER, NC State University, Raleigh, NC, United States

NUSRAT ZAHAN, NC State University, Raleigh, NC, United States

Open Access Support provided by:

NC State University

PDF Download
3696630.3734200.pdf
20 March 2026
Total Citations: 1
Total Downloads: 682

Published: 28 July 2025

Citation in BibTeX format

FSE Companion '25: 33rd ACM International Conference on the Foundations of Software Engineering
June 23 - 28, 2025
Trondheim, Norway

Conference Sponsors:
SIGSOFT

FSE Companion '25: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (June 2025)
https://doi.org/10.1145/3696630.3734200
ISBN: 9798400712760

Can the Rising Tide of Software Supply Chain Attacks Raise All Software Engineering Boats?

Laurie Williams
North Carolina State University
Raleigh, NC, USA
lawilli3@ncsu.edu

Sivana Hamer
North Carolina State University
Raleigh, NC, USA
sahamer@ncsu.edu

Nusrat Zahan
North Carolina State University
Raleigh, NC, USA
nzahan@ncsu.edu

ABSTRACT

Software organizations largely did not anticipate how the software supply chain (SSC) would become a deliberate attack vector. Attackers have moved from finding and exploiting vulnerabilities contributed by well-intentioned developers, such as log4j, to a new generation of software supply chain attacks, aggressively implanting vulnerabilities directly into dependencies available in open source. As with SolarWinds, adversaries also find their way into builds and deployments to deploy rogue software. Once implanted, these vulnerabilities become an efficient attack vector for adversaries to gain leverage at scale by exploiting the software supply chain. Software supply chain attacks have increased exponentially since 2020.

These attacks have heightened awareness among governments and software organizations worldwide of the need for software development teams to adopt good software security practices – and, in the process, good software engineering practices. This awareness has led to two US Executive Orders, the EU Cyber Resilience Act, and other actions worldwide. For example, the CEOs of software organizations who sell to the US government now need to sign a document attesting that good software security/engineering practices were used. This paper will share an overview of these actions, empirical observations in the state-of-practice in software security practice adoption, automated software practice measurement, and how these efforts can move into general software engineering.

CCS CONCEPTS

• **Software and its engineering**; • **Security and privacy**;

KEYWORDS

Software Development Practice, Software Framework, Software Security, Security Outcome, Software Supply Chain Attack

1 INTRODUCTION

Software organizations did not anticipate how the software supply chain (SSC) would become a deliberate attack vector. Attackers have moved from finding and exploiting vulnerabilities contributed by well-intentioned developers finding and exploiting vulnerabilities contributed by well-intentioned developers, such as log4j [5, 21], to actively implanting malicious vulnerabilities directly into software dependencies [50]. Recent high-profile supply chain attacks, such as SolarWinds [6, 12, 41] and XZ Utils [13, 16, 36], made headlines and directed attention towards the importance of SSC security and software security, in general. In 2024, Sonatype reported a 156% year-over-year growth of malicious packages discovered in open source software over the past ten years [42]. In the one-year period beginning November 2023, 512,847 malicious packages were discovered in open source software ecosystems [42].

> The tide of security concerns has risen so high that governments are imposing good software (security) practices on software organizations.

Most prominently, Section 4 of US Executive Order (EO) 14028 [46], issued May 12, 2021, calls for enhancing the security of software products purchased by the US government through the adoption by software developer of software security practices. The White House Office of Management and Budget (OMB) subsequently issued the MM-22-18 memo [53] (*Enhancing the Security of the Software Supply Chain through Secure Software Development Practices*) documenting requirements for software organizations that supply software to the US government to provide self-attestation of the use of secure software development practices. Similarly, the European Union (EU) released the Cyber Resilience Act (CRA) [11], which introduces mandatory cybersecurity requirements for software manufacturers, including software design and development requirements. The implications of not complying with the CRA include significant fines, the inability to sell a product in the EU, and reputational damage.

These worldwide government actions have caused a flurry of initiatives by software organizations to: implement good software security practices within their software development lifecycle; to impose good software security practices on their vendors; and to establish measurement frameworks that enable the attestation of the use of these practices.

This paper presents the perspective that the practices, the culture changes, and the measurement frameworks being forced upon organizations by governments can and will spill over to improve software engineering practice in the general case. The set of good software security practices overlaps with the set of good software engineering practices.

> The rising tide of improved software security practices can raise software engineering "boats" with intentional collaboration and communication.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the Owner/Author(s).
FSE Companion '25, June 23–28, 2025, Trondheim, Norway
© 2025 Copyright is held by the owner/author(s).
ACM ISBN 979-8-4007-1276-0/2025/06.
https://doi.org/10.1145/3696630.3734200

Security practices are actions, procedures, techniques, or other measures that an organization can adopt to reduce the vulnerability of an information system [27]. Examples of security practices would be whether a project is actively maintained, conducting code reviews before merging pull requests, and using static or dynamic analysis tools as detection measures of vulnerabilities. In this paper, we used the term "security practice", also synonymous with security control, practices, safeguards, and countermeasures [27]. *Security outcomes* are the indicators of the performance or non-performance of security practices prescribed for an information system [49, 57].

This paper will share an overview of government actions, three studies [14, 57] that provide empirical observations in the state-of-practice in and risk-based recommendations for software security practice adoption, automated software practice measurement, and how these efforts can move into general software engineering. In Section 2, we provide an overview of government and industry security frameworks. We provide an overview of current frameworks for the automatic collection of practice signals in Section 3. We present three studies we have conducted in Sections 4, 5, and 6. We complete the paper in Section 7 with our thoughts about how software security efforts can be brought into software engineering.

2 GOVERNMENT REGULATIONS AND STANDARDS, INDUSTRY FRAMEWORKS

This section summarizes government regulations and standards and industry frameworks authored to guide organizations in improving the cybersecurity of software products.

2.1 US: Executive Orders and NIST Framework

Section 4 of US EO 14028 [46], issued May 12, 2021, and the subsequent M-22-18 memo [53] set in motion the development of new standards, tools, best practices, and other guidelines to enhance SSC security. Specifically, the following were to be developed: criteria to evaluate software security; criteria to assess the security practices of developers and suppliers; and innovative tools or methods to demonstrate conformance/attestation with the use of secure practices. The framework that enumerates these secure practices is the US National Institute of Standards and Technology (NIST) SP 800-218 Secure Software Development Framework (SSDF) [26]. EO 14028 directed software producers to generate and provide evidence in the form of artifacts that demonstrate compliance with those practices.

In 2023, memo M-23-16 [54] updated M-22-18 and the date by which government agencies must only use software that is provided by software producers who can attest to complying with the Government-specified minimum secure software development practices. The date was June 2024 for critical software and September 2024 or all other software. The official attestation by these software producers involves the CEO of the company or their designee signing the Secure Software Development Attestation Form [8] that the company presently makes consistent use of a set of software security practices derived from the SSDF.

US EO 14144 [49] ordered additional actions to improve cybersecurity, including improving accountability for software providers. Section 2 of this EO focuses on operationalizing transparency and security in third-party software supply chains. Software providers use secure software development practices to reduce the number and severity of vulnerabilities in software developed and delivered. In some instances, providers of software to the Federal Government had committed to following cybersecurity practices, yet did not fix well-known exploitable vulnerabilities in their software. The US expressed a need for greater assurance that software providers are following the practices to which they attest. The EO sets forth a requirement for machine-readable secure software development attestations and high-level artifacts to validate those attestations.

2.2 Europe: Cyber Resilience Act (CRA)

The Cyber Resilience Act (CRA) [11] is a regulation introduced by the European Union in October 2024. The regulation "*aims to set the boundary conditions for the development of secure products with digital elements by ensuring that hardware and software products are placed on the market with fewer vulnerabilities and that manufacturers take security seriously throughout a product's lifecycle*" [11]. The CRA applies to organizations that place products with digital elements in the European market, with penalties for non-compliance. Several deadlines are detailed within the CRA, with full compliance expected by December 2027. The implications for the software security landscape are similar in magnitude to the General Data Protection Regulation (GDPR) in privacy [10].

The regulation also includes requirements to improve the software supply chain security. Notably, manufacturers must fix vulnerabilities and support vulnerability management in upstream open-source components. Meanwhile, open source software contributors are not responsible for helping the downstream software supply chain. Still, despite the broad implications of the CRA, 62% of respondents to a survey by OpenSSF are not familiar with or slightly familiar with the CRA [18]. Among the initiatives to help software organizations with the CRA, the Linux Foundation with OpenSSF has interviewed maintainers in essential products [2] and created a baseline of practices manufacturers can follow [20].

2.3 Other Government and Industry Frameworks

In addition to the government frameworks and regulations discussed in Sections 2.1 and 2.2, other government and industrial organizations have issued guidance (e.g., [1, 4, 26, 28, 30–32, 34]) on software development practices that enhance the security of the software supply chain. The Proactive-Software Supply Chain Risk Management Framework (P-SSCRM) [51] provides the union and restructuring of the practices outlined in the eight aforementioned frameworks [1, 4, 26, 28, 30–32, 34], the NIST SSDF [26], and the US EO 14028 [46]. The union consists of 73 practices organized into four groups: governance (G), product (P), environment (E), and deployment (D). Each practice has a unique P-SSCRM identifier.

Additionally, the Open Source Security Foundation (OpenSSF) [29], a cross-industry organization hosted at the Linux Foundation, provides a vehicle for collaboration on tools, services, training, infrastructure, and resources for securing open-source projects.

3 AUTOMATIC COLLECTION OF PRACTICE SIGNALS

This section summarizes tools that enable the automated collection of practice signals. These signals can be used for policy enforcement, attestation of the use of secure software practices, and empirical studies.

3.1 OpenSSF Scorecard

The OpenSSF Scorecard project [32] automatically generates a "security score" for open source projects to aid in risk and trust decisions on the security posture of open source projects. The OpenSSF Scorecard [32], developed by the OpenSSF [29] under the Linux Foundation, is an automated tool designed to evaluate the security posture of open-source projects. Launched in 2020, Scorecard assesses projects based on 18 security metrics, each scored on a scale from 0 to 10. The tool covers preventive and detection practices that overlap with practices included in other security frameworks (e.g., SSDF framework [26]). Each metric is assigned a risk label (Critical, High, Medium, or Low), which is used to compute an aggregate weighted score to reflect overall project security. The tool enables developers, users, and organizations to make informed decisions about the risks introduced by third-party dependencies. Scorecard results are publicly accessible via BigQuery, and the tool can also be executed directly on any GitHub or GitLab repository. The machine-checkable evaluations reduce the manual effort required for security analysis and support proactive risk management in software supply chains.

3.2 Macaron

Oracle Lab developed the Macaron tool that verifies the build integrity and dependency security of software artifacts [15]. The tool is built upon Supply Chain Levels for Software Artifacts (SLSA) [31] specification to help prevent supply chain attacks, such as credential theft and code injection. Macaron also supports user-defined policies to detect anomalies in the build process, to reduce the risk that third-party code has not been compromised.

3.3 in-toto

The *in-toto* framework allows end users to verify that each step, from creation to distribution, used in developing products follows project-defined policies [44]. Product owners define the policies in *layout* files, including allowed steps, developer keys, and expiration dates. Then, when developers carry out steps, a signed *link* metadata file is generated with the associated artifacts used. Finally, end users can verify the information in the *layout* and *link* files. The *in-toto* framework is helping software organizations to provide machine-readable attestations that can be validated, as required in regulations such as the US EO 14144 [49].

3.4 TUF

The *in-toto* framework does not address how to securely distribute, revoke, and replace the public keys used to verify the in-toto layout [44]. To address this, Samuel et al. [39] designed and implemented "The Update Framework (TUF)", a software update framework to minimize the impact of key compromise. TUF leverages mechanisms such as roles, their signatures (PKI), threshold number of signatures, file hashes, and file size to secure the system that automatically identifies and downloads updates to the software. All the information is provided as metadata files, which are version-numbered and have an expiration date. TUF is hosted by the Linux Foundation and is used in production by a growing number of companies, such as Datadog [7], IBM [17], OpenSSF [33], and the Python Packaging Index [43].

3.5 Sigstore

Sigstore [24] is a system to provide baseline artifact signing capabilities to developers to manage long-term signing keys. Sigstore follows a three-phase process: Trust Setup using TUF for root key management, a Signing Flow involving identity verification (OIDC), short-lived certificates (Fulcio), and transparency logging (Rekor), and a Verification Flow that validates identity, signature, and certificate freshness. Sigstore has quickly been adopted in Internet infrastructure, with over 2.2M signatures over critical software such as Kubernetes and Distroless [24].

4 WHAT WE HAVE LEARNED: THE ADOPTION OF SOFTWARE SECURITY PRACTICES

During 2023, we used the P-SSCRM [51] framework to conduct an empirical study of software supply chain security practice adoption among nine industry-leading software supply chain risk management initiatives in medium and large companies (the identities of the companies remain anonymous). We summarize what we have learned in this section.

For each of the nine companies, 5 to 13 practitioners were interviewed for at least one hour to assess the adoption of each of the 73 practices. In many cases, more than one practitioner was asked about each practice. A total of 61 practitioners were interviewed: 1 Chief Information Security Officer (CISO), 27 in the general area of Governance (software security group, risk management, vendor management); 23 in the Product area (architect, developer); and 10 in the areas of Environment/Deployment (DevOps, Product Security Incident Response (PSIRT)).

Based on the interviews, each practice was assigned a value for adoption in each company:

- 0: Not observed, not started
- 0.25: Beginning to emerge
- 0.50: Progress being made
- 0.75: Close to implementing
- 1.00: Task implemented, achieving task objective, exemplary implementation

Tables 1 and 2 provide the Top 10 Practices Adopted and Bottom 11 Practices Adopted, respectively. The first column of each table contains the P-SSCRM identifier for the practice. In the identifier, the Group of the practice can be found by the first letter (G=Governance, P=Practice, E=Environment, D=Deployment).

4.1 Top 10 Practices Adopted

Table 1 provides the average adoption for the Top 10 most adopted Practices. Four of the Top 10 are in the Environment Group, indicating a maturity in securing development and build environments by

Table 1: Top ten practices adopted

ID	Practice Name	Adoption %
E.3.1	Authentication	1.00
P.4.2	Automated security scanning tools	0.97
G.4.1	Role-based training	0.97
E.2.7	Build output	0.94
G.2.3	Roles and responsibilities	0.92
E.3.7	Boundary protection	0.91
G.1.2	Software license	0.89
G.2.6	Protection of information at rest	0.86
D.1.3	Vulnerability disclosure	0.86
E.3.2	Environmental separation	0.84

Table 2: Bottom eleven practices adopted

ID	Practice Name	Adoption %
E.2.6	Reproducible builds	0.03
P.5.1	SBOM Consumption	0.03
P.3.3	Require signed commits	0.08
G.1.4	Deliver provenance	0.08
P.3.5	Prevent component vetting bypass	0.14
G.1.3	Produce attestation	0.17
D.2.2	Build process monitoring	0.18
G.1.5	Deliver SBOM	0.19
E.3.9	Ephemeral credentials	0.22
E.2.3	Defensive compilation and build	0.25
P.3.2	Trusted repositories	0.25

the IT and DevOps roles. Empirical observations support that the IT and DevOps organizations in these companies had begun hardening the development and build environments before the current focus on software supply chain security.

The highly adopted Governance Practices are (1) conducting role-based security training (G.4.1) and (2) establishing roles and responsibilities for handling security within an organization (G.2.3) conducted by the Software Security Group; and (3) identifying software licenses that may conflict with an organization's policies conducted by the Business Manager role (G.1.2); and (4) protection of information at rest (G.2.6). Software organizations can get into legal risks if they utilize software that conflicts with their organization's policies. Therefore, Software Composition Analysis (SCA) tools have historically been used to check license compliance of the software components used in a product. SCA tool functionality has expanded in recent years to identify vulnerabilities in these software components and produce a Software Bill of Materials (SBOM). As such, SCA tools are playing a more significant role with the focus on software supply chain security.

The only Product Practice in the Top 10 is the use of automated security scanning tools (P.4.2), which many organizations are integrating into their CI/CD pipelines and are using to scan third-party components. Finally, the only Deployment Practice in the Top 10 is vulnerability disclosure (D.1.3). Organizations are enhancing their vulnerability disclosure programs and are understanding the implications of the bad press that comes when a vulnerability is not disclosed. In general, the Top 10 Practices were adopted by organizations before the focus on software supply chain security, and therefore, their implementation in organizations is more mature.

4.2 Bottom 11 Practices Adopted

Table 2 provides the average adoption rate for the Bottom 11 least adopted Practices. Eleven (instead of 10) are chosen due to a tie for the bottom two Practices. In the US, the Governance tasks of producing and delivering attestation (G.1.3), provenance (G.1.4), and SBOM (G.1.5) are becoming mandatory, though adoption of these Practices is low. Consuming (or finding value from) these artifacts is essentially not happening yet, based on the interviews and as shown in the SBOM consumption task (P.5.1). These Practices are generally considered compliance "checkboxes" that are not yet strictly enforced and without valuable use cases to date.

A Task that is included in the Bottom 11 is Prevent component vetting bypass (P.3.5), indicating that the limited approval processes that may be in place for components and containers may still be easily bypassed. Also low is the use of trusted repositories, such as npm or DockerHub, (P.3.2). Often developers are free to upload a component or container any source.

Three of the Environment Practices appear in the Bottom 11. The reproducible build Practice (E.2.6) has not been fully implemented by any company. This bit-by-bit comparison between two or more builds of the same product is conducted to confirm that no malicious backdoor injections have taken place during the build process. Organizations can see the value in reproducible builds, but the time and resources needed to build multiple times have kept this Practice from being implemented. Companies are also not using compiler, interpreter, and build tool features to reduce vulnerabilities, such as producing compiler warnings for vulnerable code that are treated as errors or applying obfuscation techniques (E.2.3). Finally, firms have also not adopted the practice of enhancing the security of the development environment by using ephemeral credentials (E.3.9) (i.e., randomly generated, short-lived access credentials that exist only for one session to authenticate and authorize privileged connections).

The Deployment Practice contained in the Bottom 11 list Build Process Monitoring (D.2.2) in which a firm continuously monitors the build and deployment environment to detect suspicious or unexpected activity, such as attempts to connect to suspicious endpoints during the build process.

Key Takeaway: The most-adopted software practices had been implemented by organizations before the focus on software supply chain security. Therefore, their implementation in organizations is more mature. The practices that mitigate the novel attack vectors through malicious commits of components and through the build infrastructure are in the early stages of adoption.

5 WHAT WE HAVE LEARNED: THE RELATIONSHIP BETWEEN SECURITY PRACTICE ADOPTION AND SECURITY OUTCOMES

The SSC security frameworks (Section 2) contain a long list of security practices that practitioners should adopt. Given the limited budget, time, and resources [14, 38], practitioners are often overwhelmed by the number of practices recommended in these frameworks [56]. Practitioners want to prioritize adopting security practices based on empirical evidence showing that adopting security practices improves security outcomes and mitigates security risk. In 2025, the US EO 14144 [49] strengthens the improving SSC security efforts by requiring: "***Within 240 days of the date of this order,....the Director of NIST, shall evaluate common cybersecurity practices and security outcomes that are commonly used or recommended across industry sectors.***" In this study, we aim to assist practitioners and policymakers in making informed decisions on which security practices to adopt to improve software security.

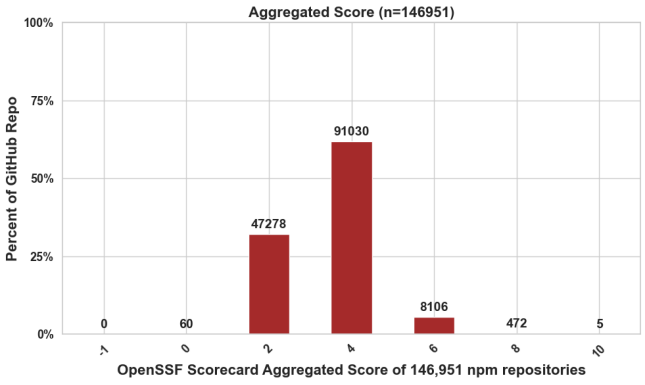

Figure 1: Distribution of OpenSSF Scorecard Aggregated Scores Across npm GitHub Repositories.

5.1 Research Overview and Key Findings

In multiple studies [55–57], we conducted large-scale empirical analysis on open-source repositories to investigate the relationship between software security practices and security outcome metrics. From these studies [55–57], we have learned both the potential and limitations of current tools for measuring security practices, revealed ecosystem-wide trends in practice adoption, and guided prioritizing security investments in OSS. In this keynote paper, we focus on the most recent study [57], which builds upon our prior works [55, 56].

We used the OpenSSF Scorecard [32] tool (see Section 3.1) to automatically measure the adoption of security practices in GitHub repositories at scale. We examined whether Scorecard security practice metrics and their aggregated score are associated with security outcome metrics. We selected two temporal vulnerability metrics that capture both exposures (Vul_Count: number of open vulnerabilities) and responsiveness (MTTR: mean time to remediate vulnerable dependencies), and one temporal software metric (MTTU: mean time to update dependencies) as security outcome metrics [57].

We have collected security practice metrics data using Scorecard [32] and security outcome metrics [32, 35] for 146,951 npm GitHub repositories. Our repository selection criteria and dataset construction source are discussed in detail in [57]. We identified which practices are strongly associated with security outcomes using regression modeling and causal inference techniques. We also explored whether these relationships are influenced by project characteristics such as size, age, and popularity.

The Scorecard aggregated score is calculated as a weighted average of individual security practice scores, with each practice weighted by the relative risk level (low, medium, high, or critical). The aggregated score helps practitioners assess the security risk of their dependencies by providing a single, interpretable measure of a project's overall security posture. Figure 1 highlights insights into the adoption of security practices across the npm ecosystem. Of the 146,951 repositories analyzed, 32.2% scored a 2, and 61.9% scored a 4, indicating more than 90% of npm repositories implemented security practices partially. Only 5.8% of repositories score above 6, and less than 0.3% achieve 8. Only five repositories achieve a perfect score of 10. The findings reveal that software organizations do not yet widely adopt software security practices and highlight the need for investment in improving the adoption of security practices across the npm ecosystem.

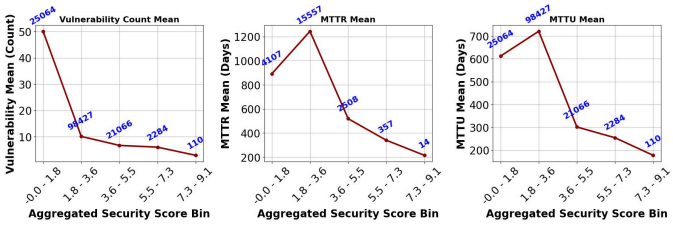

Figure 2: Security Outcome Trends and npm Repository Distribution Across Aggregated Scorecard Score Bins [57]

Figure 2 groups 146,951 npm repositories by aggregated Scorecard scores and shows mean security outcomes for each bin. As illustrated in Figure 2, there is a sharp decline in Vul_Count, MTTR, and MTTU as the aggregated Scorecard score increases, indicating improved security outcomes in repositories that adopt security practices. However, the plot only reflects an overall trend and does not control for confounding factors. We use regression analysis and causal analysis to assess the statistical relationship. We found that higher aggregated scores are associated with fewer vulnerabilities and faster MTTU, implying improved security outcomes. While regression coefficients for MTTR suggest that higher aggregated scores are associated with faster MTTR, causal analysis reveals that MTTR is also influenced by project characteristics such as size, age, contributor count, dependency count, and downloads.

We investigate the relationship between individual security practices and security outcome metrics to inform the prioritization of

practices based on their impact. Using a regression model with feature importance ranking, we find that Code-Review, Maintained status, Pinned Dependencies, and Branch Protection are the practices most strongly associated with security outcomes. Overall, vulnerability count, MTTU, and MTTR exhibit similar trends when used as security outcome metrics and the adoption of these individual practices is linked to fewer vulnerabilities and faster MTTU. However, causal analysis showed MTTR is more complex, indicating it is influenced by additional factors such as project maintainability, resource availability, and dependency complexity, which warrant further investigation into its external dependencies.

Key Takeaway: Despite evidence showing an association of security practices with security outcomes, software organizations do not yet widely adopt software security practices in the npm ecosystem.

5.2 Security Measurement: Automation Challenges and Tooling Limitations

The section outlines the challenges we encountered during our study, highlighting the limitations of automated security measurement and the complexities of interpreting Scorecard results at scale.

5.2.1 ***Metrics Detection Challenge.*** Due to Scorecard's limited detection capacity of external workflows and packaging ecosystems, metrics like Packaging and Signed-release often score -1 for 97% of repositories. Similarly, SAST and Fuzzing scores are often zero, not because the practices are not adopted at all, but because Scorecard only detects a limited set of predefined tools and configurations. These gaps lead to possible skewed distributions and underrepresentation of real-world secure software development practices.

5.2.2 ***Measurement Inconsistency.*** Many Scorecard checks rely on binary heuristics, yielding either a perfect score of 10 or a 0, which results in skewed and extreme score distributions. For instance, Binary-Artifacts penalizes all binaries equally, regardless of risk and use case, and Code-Review penalizes solo-maintainer projects even if reviews may have been done manually. Our findings reflect a broader issue in automated measurement: while simple rules aid in automated detection, such rules also can overlook real-world software development practices. Hence, contextual awareness or flexibility matters, as rigid checks can misrepresent open-source projects' true security posture and limit the resulting metrics' actionable value.

5.2.3 ***Inconsistency Across Data Sources.*** We observed inconsistencies between BigQuery data and direct Scorecard runs. Some repositories were flagged as having no vulnerabilities in BigQuery but were found to have vulnerabilities when scanned directly. This discrepancy raises concerns about data accuracy across different platforms.

5.2.4 ***Measuring Security Outcomes.*** We encountered several challenges in measuring security outcomes during the study. MTTR is challenging to compute at scale and was available for only 15% of the repositories, and fewer than half of repositories in our dataset had publicly linked Vulnerabilities. In many cases, vulnerabilities may have been silently patched or never disclosed [9, 19, 25, 37, 45, 52], making Vul_Count an incomplete and potentially unreliable indicator of overall security posture, as such vulnerabilities are not publicly reported. We tried to overcome these challenges by investigating temporal software metrics, MTTU, as a security outcome metric. Our finding shows that MTTU exhibits trends similar to Vul_Count and MTTR, making it a potential proxy metric when vulnerability and MTTR data are unavailable [57]. To more reliably assess the relationship between security practices and outcomes, future work should consider a controlled, long-term study that aligns timestamps of security practice implementation with vulnerability events. Such longitudinal tracking can help us draw more definitive conclusions about the effectiveness of secure development practices.

Key Takeaway: Automated, ecosystem-wide security adoption measurement is limited by tooling constraints and incomplete metric coverage, which hinders accurate assessment of secure software development practice.

5.3 Rethinking Automated Security Measurement

Accurately measuring secure software development practices is imperative for assessing software security posture and understanding their broader impact on the software development lifecycle. Practices such as code review, continuous integration and delivery (CI/CD), and proactive dependency management are well-established in both security and software engineering as contributors to improved software quality and maintainability. Consequently, developing reliable metrics to evaluate the adoption and effectiveness of these practices can offer insights that extend beyond security risk mitigation, providing a more comprehensive view of the software engineering discipline.

However, effectively measuring metrics at scale is a complex and unresolved challenge [3, 40, 48]. As our analysis highlights, automated tools like OpenSSF Scorecard suffer from limitations in coverage, consistency, and interpretation. A key insight from our findings is that existing automation techniques alone cannot capture the full diversity of secure software development practices. The absence of industry consensus on tooling and standards exacerbates these challenges, leaving automated tools to rely on narrow heuristics that often produce skewed or misleading results. Moreover, discrepancies between data sources (e.g., BigQuery vs. local Scorecard runs) further illustrate how data acquisition methods influence the perceived security posture.

Additionally, software is not just a technical artifact but a product of human collaboration and context [48]. As a result, software metrics often cannot be measured using a simple checklist. The unpredictability and subjectivity inherent in software development make it difficult to establish universally reliable metrics. To improve the precision and reliability of these measurements, practitioners need to refine approaches over time, incorporating methods like counted quantities, generalized prediction models, and thorough characterizations of complex systems [47].

To advance this space, we need both continuous research and broader industry coordination. Improvements in automated measurement will require consensus on what constitutes secure practices, standardization of tooling across ecosystems, and more nuanced models that account for context and intent. As NIST emphasizes, reliable measurement must evolve alongside the systems it aims to assess [47]. Without such progress, current metrics risk reinforcing narrow definitions of security that may fail to reflect real-world risk.

Key Takeaway: Measuring software metrics at scale is still a complex challenge. To improve accuracy and reliability, we need ongoing software engineering and security research, better tooling, industry-wide consensus on what constitutes secure software engineering practices and security outcomes, and more context-aware, standardized measurement approaches.

6 WHAT WE HAVE LEARNED: RISK-BASED PRACTICE ADOPTION

Software supply chain frameworks detail *what practices* software development organizations should adopt to reduce security risk. From our prior work compiling the union of 10 prominent software supply chain frameworks in P-SSCRM [51], we have found 73 practices, that ideally, should all be adopted by software organizations. However, as software supply chain security has limited resources [38], organizations would thus benefit from a *ranked priority of the mitigation practices* to adopt essential practices to reduce the risk of similar attacks. At the same time, software organizations would also benefit from knowing if *mitigation practices are missing* from the frameworks, leaving software products vulnerable to attacks and fostering a false sense of security.

In our work [14], we have systematically mapped the attack techniques in three prominent software supply chain attacks (SolarWinds, Log4j, and XZ Utils) to mitigating framework tasks. First, we qualitatively analyzed 106 reports about the attacks and manually mapped the attack techniques used in the attacks to MITRE Adversarial Tactics, Techniques and Common Knowledge (MITRE ATT&CK) [22]. MITRE ATT&CK is a widely adopted software security framework that contains attack techniques describing the specific actions used by adversaries and tactics detailing the goals of adversaries. Each attack technique has a unique identifier of the form TXXXX. Next, we systematically constructed a MITRE ATT&CK attack technique to P-SSCRM practice mapping, as no mapping between the frameworks exists. As mapping cyber-security frameworks is subjective [23], we used triangulations to find convergence through four independent, systematic, and scalable strategies. To identify missing practices, we mapped the recommendations in the 106 reports to P-SSCRM, thus transitively in the 10 reference frameworks. We scored each practice based on the number of attack techniques mitigated and the number of attacks mitigated. Finally, we ranked and selected the top ten practices in a *starter kit* list mapped to the software supply chain frameworks.

Table 3: Top ten mitigation practices in the starter kit from [14]

ID	Practice Name	Score
E.3.3	Role-based access control	90
D.2.1	System monitoring	87
E.3.7	Boundary protection	75
E.3.6	Monitor changes to configuration settings	33
E.3.11	Environmental scanning tools	27
P.5.2	Dependency update	18
P.2.1	Security design review	18
E.3.4	Information flow enforcement	15
G.2.6	Protection of information at rest	12
D.1.2	Risk-based vulnerability remediation	12

6.1 Attack Techniques

The main attack vector for **SolarWinds** was the build infrastructure. We found four stages in the incident: *Sunspot* introduction, *Sunburst* introduction, dependent exploitation, and vulnerability discovery. The adversaries leveraged 94 attack techniques, using all 14 tactics. Meanwhile, the main attack vector for **Log4j** was the dependencies. We found four stages in the incident: vulnerability introduction, vulnerability disclosure, and post-disclosure. We found 34 attack techniques and all 14 tactics leveraged in the incident. Finally, the humans were the main attack vector for **XZ Utils**. We found four stages: became maintainer, backdoor introduction, backdoor improvement, and vulnerability discovery. We found 35 attack techniques in the incident, using 11 tactics.

The unique attack techniques across all attacks are 114. Hence, attack techniques are diverse in software supply chain attacks. Additionally, the attacks are complex, with a minimum of 34 attack techniques used in an attack. Finally, we found 12 attack techniques that overlap between all three attacks analyzed, despite each attack having a different main attack vector. Among the overlapping attack techniques are: trusted relationships (T1199), supply chain compromise (T1195), obfuscated files or information (T1027), and compromise infrastructure (T1584). Research and risk reduction efforts can focus on the overlapping attack techniques.

Key Takeaway: Attack techniques used in software supply chain attacks are diverse and complex. Yet, attack techniques overlap across software supply chain attacks that can be a focus of research and risk reduction efforts.

6.2 Mitigating Practices

We found only 37 practices that mitigate attack techniques, with 34 mapping to software supply chain frameworks in P-SSCRM. Hence, less than half of the software supply chain practices in software supply chain frameworks mitigate attack techniques in attacks.

Table 3 shows the top ten practices we found in our starter kit. The first column contains the P-SSCRM identifier for the practice, followed by the practice name and the score of each practice. The

top five practices are: role-based access control (E.3.3), system monitoring (D.2.1), boundary protection (E.3.7), monitor changes to configuration settings (E.3.6), and environmental scanning tools (E.3.11). Four of the five practices are in the environment (E) group, while one is in the deployment (D) group. Meanwhile, the following top five scoring practices are: security design review (P.2.1), dependency update (P.5.2), information flow enforcement (E.3.4), protect information at rest (G.2.6), and risk-based vulnerability remediation (D.1.2). Two practices are of the product (P) group, one of the environment (E) group, one of the governance (G) group, and one of the deployment (D) group. We found at least on practice of each group in the starter kit practices. Interestingly, most starter kit practices apply to broader software security rather than being specific to the software supply chain security. Hence, we recommend that software organizations prioritize good software security practices to reduce the risk of software supply chain attacks.

Key Takeaway: Top-ranking mitigating software supply chain practices are software security practices. Hence, software organizations should prioritize good software security practices to reduce the risk of software supply chain attacks.

6.3 Missing Practices

We found three practices missing from all ten prominent software supply chain frameworks mapped in P-SSCRM. Thus, software products would still be vulnerable to software supply chain attacks even if organizations adopted all recommended practices. A summary of the three missing practices is as follows:

- *Sustainable open-source software* (G.5.5). As open-source is not a supplier, software organizations should be responsible consumers and sustainable contributors of the open-source software adopted.
- *Environmental scanning tools* (E.3.11). Automated scanning tools focused on detecting anomalies in each environment, including the development environment. The missing practice was ranked as a top-five practice in the starter kit.
- *Response partnerships* (D.1.7). Building and maintaining response partners can aid software organizations during vulnerability response.

The missing practices are not knowledge gaps, as prior work has studied similar topics, including the software engineering field. For example, there has been a plethora of research on sustainable open-source (G.5.5) within software engineering. Collaboration, a topic related to the missing practice of response partnerships (D.1.7), has also been a research topic in software engineering. Therefore, the missing practices are systematic gaps when compiling the software supply chain frameworks. Further collaboration between the fields is beneficial. The software security field can benefit from the insights from software engineering. Meanwhile, the advances in the software security field can benefit software organizations of adopting good practices.

Key Takeaway: Missing practices across software supply chain frameworks are related to topics studied in the software engineering field. Therefore, collaboration between software security and software engineering is beneficial for both.

7 THE RISING TIDE

Recent years have seen a rise in security attacks, particularly in the exponentially growing software supply chain attacks. As a result, governments in the US, Europe, and elsewhere are putting **unprecedented pressure on software organizations to adopt secure software development practices**. Organizations that do not comply with the adoption of these practices will face the implications of fines, reputational damage, and not being able to sell products to the government, which is often one of their largest customer. In the US, CEOs must sign attestation forms documenting their compliance with practice adoption. The US government is calling for automated, machine-readable attestation and high-level artifacts to accompany the signed attestation forms.

The set of software security practices overlaps with the set of software engineering practices, such as design, architectural review, code review, and testing. The culture change of adopting software security practices could spread to the adoption of other software engineering practices. Finally, the machine-readable attestation and Scorecard automation can enable measurement and measurement studies in software engineering as it has with software security.

8 ACKNOWLEDGEMENTS

This material is based upon work supported by the National Science Foundation Grant Nos. 2207008. Any opinions expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

[1] Black Duck. Building Maturity In Maturity Model (BSIMM). *https://www.blackduck.com/services/security-program/bsimm-maturity-model.html* (2025).
[2] Boehm, M., Carter, H., and Osborne, C. Pathways to cybersecurity best practices in open source: How the civil infrastructure platform, yocto project, and zephyr project are closing the gap to meeting the requirements of the cyber resilience act, 2025.
[3] Cheng, Y., Deng, J., Li, J., DeLoach, S. A., Singhal, A., and Ou, X. Metrics of security. In *Cyber defense and situational awareness*. Springer, 2014, pp. 263–295.
[4] Cloud Native Computing Foundation. Software supply chain best practices v2. *https://tag-security.cncf.io/blog/software-supply-chain-security-best-practices-v2/* (2024).
[5] Cyber Safety Review Board. Review of the December 2021 Log4j Event. https://www.cisa.gov/sites/default/files/publications/CSRB-Report-on-Log4-July-11-2022_508.pdf, 2022.
[6] Cybersecurity and Infrastructure Security Agency. Joint Statement by the Federal Bureau of Investigation (FBI), the Cybersecurity and Infrastructure Security Agency (CISA), the Office of the Director of National Intelligence (ODNI), and the National Security Agency (NSA). https://www.cisa.gov/news-events/news/joint-statement-federal-bureau-investigation-fbi-cybersecurity-and-infrastructure-security-agency-0, 2021.
[7] Datadog Engineering. Secure publication of datadog agent integrations with tuf and in-toto. *https://www.datadoghq.com/blog/engineering/secure-publication-of-datadog-agent-integrations-with-tuf-and-in-toto/* (2023).
[8] Department of Homeland Security Cybersecurity and Infrastructure Security Agency (CISA). Secure software development attestation form, 2024.
[9] Dunlap, T., Thorn, S., Enck, W., and Reaves, B. Finding fixed vulnerabilities with off-the-shelf static analysis. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)* (2023), pp. 489–505.

[10] European Union. The protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation), 2016.

[11] European Union. Horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) No 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act), 2024.

[12] FireEye. Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor. https://cloud.google.com/blog/topics/threat-intelligence/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor, 2020.

[13] Freund, A. backdoor in upstream xz/liblzma leading to ssh server compromise. https://www.openwall.com/lists/oss-security/2024/03/29/4, March 29, 2024.

[14] Hamer, S., Bowen, J., Haque, M. N., Hines, R., Madden, C., and Williams, L. Closing the Chain: How to reduce your risk of being SolarWinds, Log4j, or XZ Utils. *arXiv preprint arXiv:2503.12192* (2025).

[15] Hassanshahi, B., Mai, T. N., Michael, A., Selwyn-Smith, B., Bates, S., and Krishnan, P. Macaron: A logic-based framework for software supply chain security assurance. In *Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses* (2023), pp. 29–37.

[16] Hat, R. Urgent security alert for fedora linux 40 and fedora rawhide users. https://www.redhat.com/en/blog/urgent-security-alert-fedora-40-and-rawhide-users, March 29, 2024.

[17] Kubernetes, and IBM. Building an image trust service on kubernetes with notary and tuf. https://kubernetes.io/case-studies/ibm/, n.d. Accessed: 2024-09-26.

[18] Lawson, A., and Hendrick, S. Unaware and uncertain: The stark realities of cyber resilience act readiness in open source, 2025.

[19] Li, Z., Zou, D., Xu, S., Jin, H., Qi, H., and Hu, J. Vulpecker: an automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd annual conference on computer security applications* (2016), pp. 201–213.

[20] Linux Foundation. Open source project security baseline. *https://github.com/ossf/security-baseline* (2021).

[21] Log4J. Apache log4j security vulnerabilities, 2021.

[22] MITRE. MITRE ATT&CK. https://attack.mitre.org/, 2024.

[23] National Institute of Standards and Technology. NIST IR 8477 Mapping Relationships Between Documentary Standards, Regulations, Frameworks, and Guidelines: Developing Cybersecurity and Privacy Concept Mappings. https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8477.pdf, 2024.

[24] Newman, Z., Meyers, J. S., and Torres-Arias, S. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (2022), pp. 2353–2367.

[25] Nguyen-Truong, G., Kang, H. J., Lo, D., Sharma, A., Santosa, A. E., Sharma, A., and Ang, M. Y. Hermes: Using commit-issue linking to detect vulnerability-fixing commits. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (2022), IEEE, pp. 51–62.

[26] NIST. Secure Software Development Framework (SSDF). *https://csrc.nist.gov/projects/ssdf* (2022). Accessed: February 21, 2025.

[27] NIST. Security Controls - NIST Glossary, 2024. Accessed: February 19, 2025.

[28] NIST. SP 800-161 Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations. *https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-161r1-upd1.pdf* (2024).

[29] Open Source Security Foundation. Openssf: Open source security foundation, 2024. Accessed: 2025-04-24.

[30] OpenSSF. Software Supply Chain Consumption Framework (S3C2F). *https://github.com/ossf/s2c2f* (2022).

[31] OpenSSF. Supply chain Levels for Software Artifacts (SLSA). *https://slsa.dev/* (2022).

[32] OpenSSF. OpenSSF Scorecard - Security health metrics for Open Source, 2025.

[33] OpenSSF. Repository service for tuf (rstuf). https://openssf.org/projects/repository-service-for-tuf/, n.d. Accessed: 2024-09-26.

[34] OWASP. Software Component Verification Standard (SCVS) . *https://owasp-scvs.gitbook.io/scvs/* (2020).

[35] Rahman, I., Zahan, N., Magill, S., Enck, W., and Williams, L. Characterizing dependency update practice of npm, pypi and cargo packages. *arXiv preprint arXiv:2403.17382* (2024).

[36] Russ Cox. Timeline of the xz open source attack. https://research.swtch.com/xz-timeline, 2024.

[37] Sabetta, A., and Bezzi, M. A practical approach to the automatic classification of security-relevant commits. In *2018 IEEE International conference on software maintenance and evolution (ICSME)* (2018), IEEE, pp. 579–582.

[38] Sammak, R., Rotthaler, A. L., Ramulu, H. S., Wermke, D., and Acar, Y. Developers' approaches to software supply chain security: An interview study. In *Proceedings of the 2024 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses* (2023), pp. 56–66.

[39] Samuel, J., Mathewson, N., Cappos, J., and Dingledine, R. Survivable key compromise in software update systems. In *Proceedings of the 17th ACM conference on Computer and communications security* (2010), pp. 61–72.

[40] Scala, N. M., Reilly, A. C., Goethals, P. L., and Cukier, M. Risk and the five hard problems of cybersecurity. *Risk Analysis 39*, 10 (2019), 2119–2126.

[41] SolarWinds. SolarWinds Security Advisory. https://www.solarwinds.com/sa-overview/securityadvisory, 2021.

[42] Sonatype. State of the software supply chain: A decade of data, 2024.

[43] Stufft, D., Cappos, J., and Kuppusamy, T. K. Pep 458 – secure pypi downloads with tuf. https://peps.python.org/pep-0458/#pypi-and-tuf-metadata, 2014. Accessed: 2024-09-26.

[44] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)* (2019), pp. 1393–1410.

[45] Wang, X., Sun, K., Batcheller, A., and Jajodia, S. Detecting" 0-day" vulnerability: An empirical study of secret security patch in oss. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)* (2019), IEEE, pp. 485–492.

[46] White House. Executive order 14028 on improving the nation's cybersecurity. *https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity* (2021).

[47] White House. Federal Cybersecurity Research and Development Strategic Plan 2023. *https://www.whitehouse.gov/wp-content/uploads/2024/01/Federal-Cybersecurity-RD-Strategic-Plan-2023.pdf* (2023).

[48] White House. BACK TO THE BUILDING BLOCKS: A PATH TOWARD SECURE AND MEASURABLE SOFTWARE. *https://www.whitehouse.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf* (2024).

[49] White House. Executive order 14144 on strengthening and promoting innovation in the nation's cybersecurity, January 16 2025. Accessed: 2025-04-28.

[50] Williams, L., Benedetti, G., Hamer, S., Paramitha, R., Rahman, I., Tamanna, M., Tystahl, G., Zahan, N., Morrison, P., Acar, Y., Cukier, M., Kästner, C., Kapravelos, A., Wermke, D., and Enck, W. Research directions in software supply chain security. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025). Just Accepted.

[51] Williams, L., Migues, S., Boote, J., and Hutchison, B. Proactive Software Supply Chain Risk Management Framework (P-SSCRM) Version 1. *https://arxiv.org/pdf/2404.12300* (2024).

[52] Xu, Z., Chen, B., Chandramohan, M., Liu, Y., and Song, F. Spain: security patch analysis for binaries towards understanding the pain and pills. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)* (2017), IEEE, pp. 462–472.

[53] Young, S. D. M-22-18 enhancing the security of the software supply chain through secure software development practices. *https://www.whitehouse.gov/wp-content/uploads/2022/09/M-22-18.pdf* (2022).

[54] Young, S. D. M-23-16 update to memorandum m-22-18, enhancing the security of the software supply chain through secure software development practices. *https://www.whitehouse.gov/wp-content/uploads/2023/06/M-23-16-Update-to-M-22-18-Enhancing-Software-Security.pdf* (June 9, 2023).

[55] Zahan, N., Kanakiya, P., Hambleton, B., Shohan, S., and Williams, L. Openssf scorecard: On the path toward ecosystem-wide automated security metrics. *IEEE Security & Privacy 21*, 6 (2023), 76–88.

[56] Zahan, N., Shohan, S., Harris, D., and Williams, L. Do software security practices yield fewer vulnerabilities? In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (2023), IEEE, pp. 292–303.

[57] Zahan, N., and Williams, L. Prioritizing security practice adoption: Empirical insights on software security outcomes in the npm ecosystem. *arXiv preprint arXiv:2504.14026* (2025).